

Django pdf 2

Практическое задание №1	3
Теоретическая часть	3
Практическая часть	7
Задания	20
Контрольные вопросы	20

Практическое задание №1

Class-based views(базовые классы представлений)

Теоретическая часть

Django Class-Based Views (CBVs) – это мощный инструмент, предоставляемый Django, который позволяет создавать гибкие, масштабируемые и переиспользуемые представления на основе классов. В отличие от функциональных представлений (Function-Based Views, FBVs), CBVs используют классы, которые дают больше возможностей для настройки и наследования.

Основные принципы CBVs:

- Классы вместо функций: CBVs используют объектно-ориентированный подход, позволяя логически разделять код.
- Миксины: CBVs поддерживают наследование и повторное использование кода с помощью миксинов.
- Шаблоны проектирования: Django предоставляет готовые базовые классы (generic views), которые решают стандартные задачи, такие как отображение списка объектов, создание формы и т.д.

Основные базовые классы CBVs:

View

Это базовый класс для всех CBVs. Он обрабатывает HTTP-запросы и определяет методы, такие как `get`, `post`, `put`, `delete` для разных типов запросов.

TemplateView

Используется для рендеринга статического шаблона без передачи сложной логики.



src/post/views.py

```
from django.views.generic import TemplateView

class HomePageView(TemplateView):
    template_name = "home.html"
```

1. ListView

Используется для отображения списка объектов из базы данных.



src/post/views.py

```
from django.views.generic import ListView
from .models import Post

class PostListView(ListView):
    model = Post
    template_name = "post_list.html"
    context_object_name = "posts"
```

2. DetailView

Предназначен для отображения одного объекта.



```
src/post/views.py  
  
from django.views.generic import DetailView  
from .models import Post  
  
class PostDetailView(DetailView):  
    model = Post  
    template_name = "post_detail.html"  
    context_object_name = "post"
```

3. **CreateView, UpdateView, DeleteView**

Используются для работы с формами. Например, создание нового объекта:



```
src/post/views.py  
  
from django.views.generic.edit import CreateView  
from .models import Post  
  
class PostCreateView(CreateView):  
    model = Post  
    template_name = "post_form.html"  
    fields = ['title', 'content']  
    success_url = '/'
```

Преимущества CBVs:

- Повторное использование кода через наследование.

- Четкая структура, особенно для сложных приложений.
- Использование готовых generic views сокращает время разработки.

Недостатки CBVs:

- Сложнее понять новичкам.
- Иногда требуются нестандартные модификации, которые могут усложнить код.

Практическая часть

В django 2.pdf мы будем реализовывать менеджер паролей. Для начало реализуем простой CRUD(create read update delete) с помощью class based views (базовые классы представлений). Создаем проект с приложением password_manager.

Создаем модель EntryPassword

```
src/password_manager/models/entry_password.py

class EntryPassword(models.Model):
    website_name = models.CharField(max_length=100)
    website_url = models.CharField(max_length=255)

    username = models.CharField(max_length=255)
    password = models.CharField(max_length=255)

    created_at = models.DateTimeField(auto_now_add=True)

    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return f"{self.website_url}"
```

Теперь заходим во views и реализуем базовые классы представлений. Для начало реализуем простой вывод html страницы. Создаем класс HomeView и наследуемся от класса TemplateView.

Затем пишем атрибут класса template_name в котором пишем название html файла.



```
src/password_manager/views/base.py
```

```
from django.views.generic import TemplateView
```

```
class HomeView(TemplateView):  
    template_name = "home.html"
```

Создаем ссылку(path), это делается, как обычно, но теперь когда пишем класс, то нам нужно использовать метод `as_views()`.



```
src/password_manager/urls.py
```

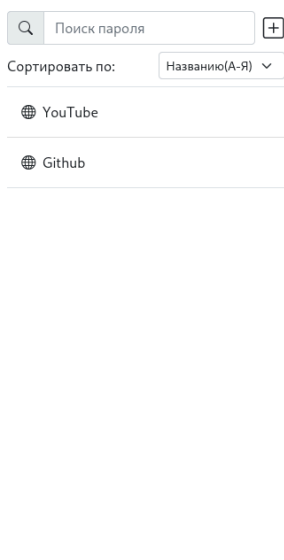
```
urlpatterns = [  
    path('', HomeView.as_view()),  
]
```

После этих действий будет отображаться страница, который вы передали в атрибут класса `template_name`. Теперь будет выводить данные о наших паролях с помощью класса представлений. Для вывода данных нужно переопределить метод

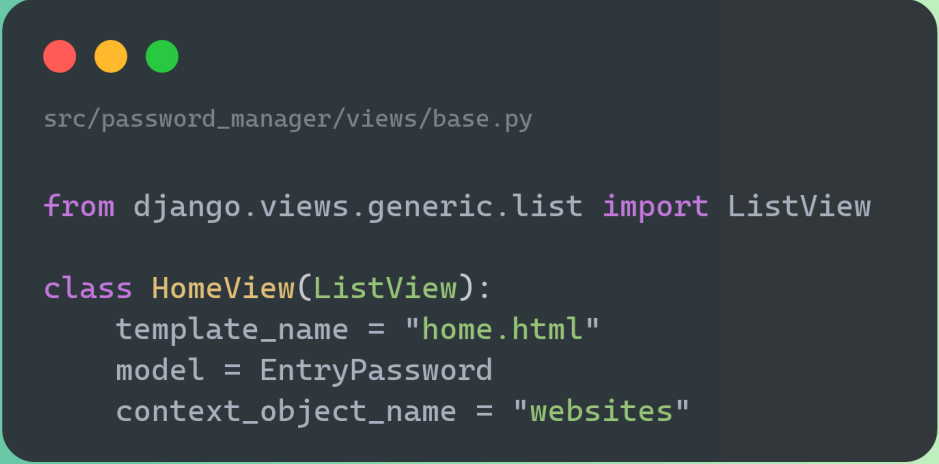
`get_context_data`, который передает данные в html шаблон(как аргумент `context` в функции `render`).



С помощью класса `super()` активируем метод с родительского класса (`TemplateView`), чтобы сохранить прошлые переменные(например `request`), который отправляется для рендера html страницы. Записываем результат в переменную, затем добавляем ключи и значение в данную переменную. Теперь можно будет воспользоваться переменной `websites`, чтобы вывести все сайты с сохранёнными данными авторизации.



Немного изменим наш класс `HomeView`, заменяем родительский класс `TemplateView` на `ListView`. С помощью класса `ListView` можно вывести все записи без переопределения метода `get_context_data`. Добавляем два атрибута класса `model` и `context_object_name`.



```
src/password_manager/views/base.py
```

```
from django.views.generic.list import ListView
```

```
class HomeView(ListView):  
    template_name = "home.html"  
    model = EntryPassword  
    context_object_name = "websites"
```

Атрибут класса `model` заполняем моделью, которую хотим использовать в html шаблоне, а атрибут класса `context_object_name` пишем название переменной для использования в шаблоне(изначально в `ListView` нет этих атрибутов, они наследуются от класса `MultipleObjectMixin`, который будет использоваться во многих классах в дальнейшем. Чтобы узнать родительские классы нужно использовать документацию или заходить в исходный код).

Реализуем вывод определенной записи. Для вывода записи определенной записи будет использовать базовый класса `DetailView`, который предназначит создание страницы по определенному идентификатору.

Создаем класс `WebSiteDataDetailView`, и наследуемся от класса `DetailView`. Добавляем те же атрибуты класса `model`, `template_name` и `context_object_name`.

```
src/password_manager/views/base.py
from django.views.generic.detail import DetailView

class WebSiteDataDetailView(DetailView):
    template_name = "website_data_detail.html"
    model = EntryPassword
    context = 'website'
```

Затем создаем путь в urls.py (int - это тип данных, который будет использовать, а pk(id) это поле из модели).

```
src/password_manager/urls.py
urlpatterns = [
    path('', HomeView.as_view(), name="home"),
    path('<int:pk>', WebSiteDataDetailView.as_view(), name="website_data_detail"),
]
```

Создаем ссылку для перехода на страницу.

```
src/templates/item_data_website.html
<a href="{% url 'website_data_detail' website.id %}" class="list-group-item list-group-item-action py-3 lh-tight">
```

И теперь при нажатии на ссылку мы будем переходить на отдельную страницу, но переход на отдельную страницу не подходит нашему макету. В нашем макете есть пустое пространство справа, которое не используется. Реализуем рендер html шаблона в правой области нашей страницы без перезагрузки страницы. Для реализации данных действия используется javascript, который

отправляет запрос на сервер(например django), а сервер возвращает какие-либо данные(например html-шаблон с заполненными данными), затем с помощью javascript вставляем(например [innerHTML](#) или [outerHTML](#)) их без перезагрузки. В данном проекте мы будем использовать библиотеку htmx для упрощения отправки запросов и вставки.

Добавляем в базовый шаблон cdn подключение [htmx](#).

```
src/templates/base.html
<script src="https://unpkg.com/htmx.org@2.0.4" integrity="sha384-HGfztotfotfshcF7+8n44JQL2oJmowVChPTg485+jvZoztPFvaD79OC/LTtG6dMp+" crossorigin="anonymous"></script>
```

Добавляем новые атрибут для тега <a> для рендера шаблона без перезагрузки страницы.

```
src/templates/item_data_website.html
<a hx-get="{% url 'website_data_detail' website.id %}"
  class="list-group-item list-group-item-action py-3 lh-tight"
  hx-target="#detail_data"
  hx-trigger="click"
  hx-swap="innerHTML"
  href="#"
>
```

hx-get – в данном атрибуте пишется ссылка, куда будет отправлен запрос(также есть hx-post, hx-delete и тд)

hx-target – в данному атрибуте пишем, элемент к которому будем обращаться(это реализуется через id или класс)

hx-trigger – в данному атрибуте при какой действии будет отправляться запрос

hx-swap - в данный атрибут пишем тип замены относительно цели запроса

Теперь при нажатии одного элемента из списка сохранённых паролей, будет загружаться детальные данные(не забудьте присвоить id элементу, куда будет вставляться html шаблон).

На данный момент реализованно только чтение записей. Чтобы реализовать функционал создание, удаление и обновление. В django есть базовые классы для реализации данного функционала – [CreateView](#), [UpdateView](#) и [DeleteView](#). Для начало изменим базовые форму django, чтобы у все input был присвоен один класс в

форме. Создаем в папке проекта(где находится файла settings) папку base, затем в папке создаем файл form.py. В файле создаем класс BaseForm, который будет наследовать от класса ModelForm. Переопределяем метод `__init__`.

```
src/core/base/form.py
```

```
from django import forms
```

```
class BaseForm(forms.ModelForm):  
    def __init__(self, *args, **kwargs):  
        super().__init__(*args, **kwargs)  
        for field in self.fields.values():  
            field.widget.attrs.update({'class': 'form-control'})
```

В методе `__init__` мы проходимся по всем поля, и каждому добавляем класс. Затем заходим в файл settings.py, и добавляем путь для нашего класс в переменную `DEFAULT_FORM_CLASS`.

```
src/core/settings.py
```

```
DEFAULT_FORM_CLASS = 'core.base.form.BaseForm'
```

Создаем форму для модели EntryPassword

```
src/password_manager/forms/entry_password_form.py

class EntryPasswordForm(forms.ModelForm):

    class Meta:
        model = EntryPassword
        fields = ['website_name', 'website_url', 'username', 'password']
```

Затем создаем класса `WebSiteDataCreate`, который будет наследоваться от `CreateView`. В данном классе пишем атрибут `form_class` в котором помещаем нашу форму, также заполняем атрибуты `template_name`, `model` и `success_url`.

```
src/password_manager/views/base.py

class WebSiteDataCreateView(CreateView):
    template_name = "website_form_create.html"
    model = EntryPassword
    form_class = EntryPasswordForm
    success_url = '/'
```

В шаблоне автоматически будет переменная `form` для вывода полей. Реализуем рендер формы. Добавляем в кнопку `htmx` атрибуты, и также будет заменять контент в `div` с `id = detail_data`.



Нажимаем ссылку(тег `<a>`) и появляется html шаблон с формой, который мы передали.

A screenshot of a web application interface. On the left, there is a sidebar with a search bar labeled 'Поиск пароля' and a dropdown menu for sorting by 'Названию(А-Я)'. Below the search bar, there are two items: 'YouTube' and 'Github'. The main content area is titled 'Менеджер паролей'. It contains a form with a globe icon in a box, followed by input fields for 'Адрес веб-сайта', 'Логин / Электронная почта', and 'Пароль'. At the bottom of the form are two buttons: 'Сохранить' (blue) and 'Отменить' (red).

Теперь, чтобы отправить форму мы также добавляем атрибуты в кнопку button, только вместо `hx-get`, пишем `hx-post` для отправки post-запроса. В атрибут `hx-target` указываем id `detail_data`, чтобы его заменить(django сам сериализует html шаблон и возьмет нужные данные).

Еще добавим кнопку, чтобы отменить создание запись. В кнопке отмена будем отправлять get запрос на ссылку главной страницы и заменять весь body(у тега body есть id =app)

```
src/templates/website_form_create.html
```

```
<div>
  <button
    hx-post="{% url 'website_data_create' %}"
    hx-target="#detail_data"
    hx-swap="innerHTML"
    type="button" class="btn btn-primary"
  >Сохранить</button>

  <button
    hx-get="{% url 'home' %}"
    hx-target="#app"
    hx-swap="innerHTML"
    type="button" class="btn btn-danger"
  >Отменить</button>
</div>
```

Создаем новую запись, форма не заменяется на запись, а также заменяется на такую же форму. Исправляем данный функционал, переопределяем метод `form_valid` (Метод `form_valid` сохраняет запись, и записывает ту запись в атрибут `object`, затем отправляет ответ на клиент). Используем класс `super`, чтобы сохранить прошлый функционал и добавим свой, затем с помощью функции `render` возвращаем html шаблон с отдельной записью и контекстом. Теперь при создании новой записи у нас будет сразу показываться новая запись.


```
src/password_manager/views/base.py

def form_valid(self, form):
    response = super().form_valid(form)
    return render(self.request, "website_data_detail.html", context={
        'website': self.object
    })
```

Создаем класс для реализации обновления записи. Пишем класс `WebSiteDataUpdateView` и наследуемся от класса `UpdateView`. Заполняем атрибуты `template_name`, `model`, `form_class`, `context_object_name` и `success_url`. Затем также переопределяем метод `form_valid` как в классе `WebSiteDataCreateView`.

```
src/password_manager/views/base.py

class WebSiteDataUpdateView(UpdateView):
    template_name = "website_form_update.html"
    model = EntryPassword
    form_class = EntryPasswordForm
    context_object_name = 'website'
    success_url = '/'

    def form_valid(self, form):
        response = super().form_valid(form)
        return render(self.request, "website_data_detail.html", context={
            'website': self.object
        })
```

Создаем путь для `WebSiteDataUpdateView`.

```
src/password_manager/urls.py

urlpatterns = [
    path('', HomeView.as_view(), name="home"),
    path('<int:pk>', WebSiteDataDetailView.as_view(), name="website_data_detail"),
    path('create', WebSiteDataCreateView.as_view(), name="website_data_create"),
    path('update/<int:pk>', WebSiteDataUpdateView.as_view(), name="website_data_update"),
]
```

Добавляем в кнопку редактирование htmx атрибуты.

```
src/templates/website_data_detail.html

<a href="#"
    hx-get="{% url 'website_data_update' website.id %}"
    hx-target="#detail_data"
    hx-trigger="click"
    hx-swap="innerHTML"
```

И также нужно добавить атрибуты у кнопок формы(используем пост запрос, потому что django при других запросов ожидает csrf токен в заголовке запроса)

```
src/templates/website_form_update.html
```

```
<div>
    <button
        hx-post="{% url 'website_data_update' website.id %}"
        hx-target="#detail_data"
        hx-swap="innerHTML"
        type="button" class="btn btn-primary"
    >Сохранить</button>

    <button
        hx-get="{% url 'website_data_detail' website.id %}"
        hx-target="#detail_data"
        hx-swap="innerHTML"
        type="button" class="btn btn-danger"
    >Отменить</button>
</div>
```

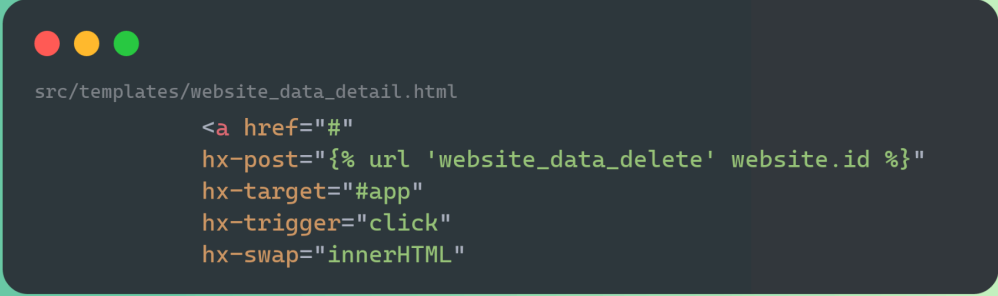
Теперь отмена будет заново возвращать в запись, которую мы хотели изменить.

Создаем класс `WebSiteDataDeleteView` для удаления записи, который наследуется от класса `DeleteView`. Заполняем атрибуты `model` и `success_url`.

```
src/password_manager/views/base.py
```

```
class WebSiteDataDeleteView(DeleteView):
    model = EntryPassword
    success_url = '/'
```

Также создаем путь для использования нашего класса. И заполняем кнопку удаление, в ней мы также будем отправлять post-запрос для удаления записи.

A code editor window with a dark background and rounded corners, set against a light green background. The window has three colored window control buttons (red, yellow, green) in the top-left corner. It displays HTML code for a delete button.

```
src/templates/website_data_detail.html  
  
<a href="#"  
  hx-post="{% url 'website_data_delete' website.id %}"  
  hx-target="#app"  
  hx-trigger="click"  
  hx-swap="innerHTML"
```

Задания

1. Уберите отображения пароля, а также добавьте кнопку, которая показывала пароль. При нажатие на кнопку должен отправляться запрос на сервер, что взять пароль.
2. Реализуйте поиск и фильтрацию без перезагрузки страницы

Контрольные вопросы

1. Какие преимущества использования CBV(базовые классы представления) по сравнению с FBV(базовые функции представления)?
- 2.